

终极期中做题蛆版复习资料

下面这份按“考算法设计题怎么写”来整理，不是单纯知识点总结。你现在最需要背的是：**题型识别 → 算法模板 → 正确性证明模板 → 复杂度分析模板**。

你这门课前半部分的主线很清楚：稳定匹配训练“定义好解与证明稳定性”，图训练“建模与遍历/结构识别”，贪心训练“局部选择为什么安全”，分治训练“递归拆解与高效合并”。你的资料里也反复强调：算法设计不是先套技巧，而是先形式化问题、识别结构、选择范式、证明正确性。

0. 算法设计题通用答题模板

考试遇到设计算法题，不要一上来写代码。标准答案通常要有四块：

1. 建模与核心想法

先写：

将问题抽象为.....
维护.....
使用.....算法/策略。
关键观察是.....

例如：

- “将每个人的出生、死亡时间建成有向图中的两个节点。”
- “将格子扩展为两种状态：未拆墙、已拆墙。”
- “按结束时间排序，每次选最早结束的区间。”
- “递归求左右两半，关键是在线性时间处理跨半部分。”

这一步很重要，因为很多题考的不是现成算法，而是 **把题目翻译成已知结构**。

2. 伪代码

伪代码不需要像 Python 一样细，但必须明确：

- 输入是什么；
- 初始化什么变量；
- 循环/递归如何推进；
- 何时返回；
- 数据结构是什么。

3. 正确性证明

常见证明结构：

模板 A：循环不变量

适合 BFS、Dijkstra、Gale-Shapley、拓扑排序。

写法：

证明在每一轮循环开始/结束时，不变量 I 成立。
初始时成立。
若某轮前成立，执行一步后仍成立。
算法终止时，由不变量推出答案正确。

模板 B：归纳法

适合分治、递推、BFS 层、递归算法。

写法：

对问题规模 n 归纳。
基础情况显然。
假设规模小于 n 的子问题都正确。
证明当前算法对子问题递归正确，且合并步骤正确。
因此规模 n 的问题正确。

模板 C：反证法

适合稳定性、拓扑排序、有向环、最短路安全性。

写法：

假设算法输出不正确。
则存在某种违反性质的对象，例如 blocking pair、同一层边、非法时间顺序、跨割更优路径。
根据算法选择规则推出矛盾。

模板 D：贪心证明

贪心题最常用三类：

1. **stays ahead**: 证明贪心每一步都不比任意最优解落后;
2. **exchange argument**: 把任意最优解一步步交换成贪心解, 且不变差;
3. **structural lower bound**: 先证明任何解都至少需要 L , 再证明贪心达到 L 。

你的贪心资料里也明确总结了这三类证明语言: stays ahead、exchange argument 和 structural proof。

4. 复杂度分析

常见写法:

每个点最多入队一次, 每条边最多检查一次, 所以 $O(m + n)$ 。
先排序 $O(n \log n)$, 后线性扫描 $O(n)$, 所以总复杂度 $O(n \log n)$ 。
递推式为 $T(n) = 2T(n/2) + O(n)$, 所以 $T(n) = O(n \log n)$ 。
每轮扫描所有 m 条边, 共 n 轮, 所以 $O(mn)$ 。

1. 稳定匹配 Stable Matching

1.1 必须掌握的重点

稳定匹配的核心不是“最优值”, 而是 **没有 blocking pair**。一个完美匹配若不存在一对当前没配在一起、但双方都更喜欢彼此的参与者, 则称为稳定匹配。你的资料里也强调: 稳定匹配追求的**不是**总幸福最大, 而是系统中不存在能私下双赢的一对人。

必须掌握:

1. matching、perfect matching、stable matching、blocking pair 的定义;
 2. Gale-Shapley 算法;
 3. GS 的三个证明: 终止性、完美性、稳定性;
 4. proposer-optimal / receiver-pessimal 的直觉;
 5. 谁提出申请会影响结果;
 6. 接收方可能通过说谎改善结果;
 7. ties 情况下 strong stability 和 weak stability 的区别;
 8. 会把别的问题规约成 stable matching, 比如船舶停靠问题。
-

1.2 必考题型 1: 写 Gale-Shapley 算法并证明稳定

题型特征

题目通常说:

有 n 个学生和 n 个医院，每个人都有偏好列表，设计一个稳定匹配算法并证明正确。

或者：

运行 Gale-Shapley，分析复杂度。

必背伪代码：Gale-Shapley

下面用 A 表示提出方， B 表示接受方。

算法 Gale-Shapley(A, B)

输入：

$A = \{a_1, \dots, a_n\}$, $B = \{b_1, \dots, b_n\}$

每个 a 对所有 b 有偏好表 $\text{prefA}[a]$

每个 b 对所有 a 有偏好表 $\text{prefB}[b]$

预处理：

对每个 b 和每个 a ，计算 $\text{rankB}[b][a]$

$\text{rankB}[b][a]$ 越小，表示 b 越喜欢 a

初始化：

对每个 a ：

$\text{matchA}[a] = \text{NIL}$

$\text{next}[a] = 1$ # a 下一次求婚的对象在 $\text{prefA}[a]$ 中的位置

对每个 b ：

$\text{matchB}[b] = \text{NIL}$

$\text{Free} =$ 所有 A 中自由的 a

循环：

当 Free 非空时：

 取出一个自由的 a

$b = \text{prefA}[a][\text{next}[a]]$

$\text{next}[a] = \text{next}[a] + 1$

 如果 $\text{matchB}[b] == \text{NIL}$ ：

$\text{matchA}[a] = b$

$\text{matchB}[b] = a$

 否则：

$a_{\text{old}} = \text{matchB}[b]$

 如果 $\text{rankB}[b][a] < \text{rankB}[b][a_{\text{old}}]$ ：

```
# b 更喜欢新来的 a
matchA[a_old] = NIL
将 a_old 加入 Free

matchA[a] = b
matchB[b] = a
```

否则:

```
# b 更喜欢原来的 a_old
matchA[a] = NIL
将 a 加入 Free
```

输出:

matchA, matchB

正确性证明模板

1. 终止性

每次循环都会发生一次新的求婚。

每个 a 不会向同一个 b 求婚两次，所以总求婚次数最多是 n^2 。

因此算法一定终止。

2. 完美性

假设算法终止后存在某个 a 未匹配。

由于算法终止， a 不可能还有没求过婚的 b ，所以它已经向所有 b 求过婚。

那么每个 b 都至少收到过一次求婚。

而接受方 b 一旦收到过求婚，就会一直保持匹配状态，只会换更喜欢的人，不会重新变成空。

所以所有 b 都被匹配。

由于两边人数相同，这意味着所有 a 也应被匹配，矛盾。

因此输出是 perfect matching。

3. 稳定性

假设输出匹配 M 不稳定。

则存在 blocking pair (a, b) ，满足：

- a 更喜欢 b 胜过当前匹配 $M(a)$ ；
- b 更喜欢 a 胜过当前匹配 $M(b)$ 。

由于 a 是按偏好从高到低求婚的，而 a 更喜欢 b 胜过 $M(a)$ ，所以 a 一定曾经向 b 求过婚。

当 a 向 b 求婚时：

- 若 b 拒绝了 a ，说明 b 当时已有更喜欢的人；
- 若 b 暂时接受了 a ，后来又抛弃 a ，说明来了更喜欢的人。

而 b 的匹配对象只会越来越好，所以最终的 $M(b)$ 一定不差于 a 。

这与 b 更喜欢 a 胜过 $M(b)$ 矛盾。

所以不存在 blocking pair，匹配稳定。

复杂度模板

预处理 $rankB$ 需要 $O(n^2)$ 。

总求婚次数最多 n^2 ，每次求婚处理 $O(1)$ 。

所以总时间复杂度：

$O(n^2)$

空间复杂度：

$O(n^2)$

1.3 必考题型 2：偏好有并列 ties

题型特征

题目会给出：

- strong instability；
- weak instability；

让你判断是否总存在稳定匹配。

你的习题资料中总结过：strong instability 要求双方都严格更喜欢，因此可以任意打破平局后运行 GS；若输出中存在强不稳定对，则它在打破平局后的严格实例中仍是 blocking pair，矛盾。weak instability 则不一定能避免。

必背结论

稳定性类型	定义	是否总存在
strong instability	双方都严格更喜欢彼此	总存在
weak instability	一方严格更喜欢，另一方不反对或也喜欢	不一定存在

strong stability 证明模板

算法：

1. 把所有并列偏好任意打破；
2. 得到严格偏好表；
3. 运行 Gale-Shapley；
4. 输出匹配。

证明：

假设输出存在 strong instability (s, h) 。

由于 strong instability 要求双方严格更喜欢，所以打破平局不会改变这两个严格偏好关系。

因此 (s, h) 在严格偏好实例中也是 blocking pair。

但 GS 输出稳定匹配，矛盾。

weak stability 反例模板

构造 $n = 2$ ：

学生：

- s_1 对 h_1, h_2 无差异。

医院：

- $h_1 : s_1 > s_2$
- $h_2 : s_1 > s_2$

两个完美匹配：

1. 若 $s_1 - h_1, s_2 - h_2$ ，则 (s_1, h_2) 是 weak instability；

2. 若 $s_1 - h_2, s_2 - h_1$, 则 (s_1, h_1) 是 weak instability。

所以不总存在无 weak instability 的完美匹配。

1.4 必考题型 3：把现实问题规约成稳定匹配

典型例子：船舶停靠问题

题型特征：

有 n 个对象和 n 个位置，需要安排一一匹配，并且要避免某种冲突。证明一定存在安排。

这类题不要硬想新算法，优先考虑能不能构造偏好表，然后调用 GS。

船舶停靠问题的构造是：

- 船按访问港口时间从早到晚排序；
- 港口按船到达时间从晚到早排序；
- 运行稳定匹配；
- 船停在匹配到的港口。

参考答案中也是这样构造：船更喜欢更早访问的港口，港口更喜欢更晚访问自己的船，然后利用 blocking pair 矛盾证明安全。

规约题证明模板

1. **构造偏好表**：说明两边是谁，偏好如何定义。
2. **运行 GS**：得到稳定匹配。
3. **把匹配翻译回原问题解**。
4. **反证正确性**：

假设原问题解不合法。

那么存在某种冲突。

根据偏好表定义，这个冲突恰好说明某一对对象互相更喜欢彼此。

于是形成 blocking pair。

这与稳定匹配矛盾。

所以解合法。

2. 图 Graphs

2.1 必须掌握的重点

图的核心是建模。资料中强调：当问题本质是“对象之间的关系”时，就把对象抽象为点，把关系抽象为边，然后在图上做算法。图题训练的是建模能力、表示能力和搜索/结构识别能力。

必须掌握：

1. 图的基本表示：邻接表、邻接矩阵；
2. BFS：无权最短路、层次结构、连通分量；
3. DFS：遍历、连通性、递归探索；
4. 二分图判定；
5. DAG 与拓扑排序；
6. 有向图可达性与强连通判定；
7. 图建模题：约束一致性、计数路径、状态扩展；
8. 带权图：Dijkstra、MST、瓶颈路径。

2.2 必考题型 1：BFS 求无权最短路/连通分量

BFS 的关键性质是：第 j 层 L_j 恰好是距离源点 s 为 j 的所有节点。因此 BFS 不只是遍历，它天然给出无权图的最短路径长度。

必背伪代码：BFS

算法 BFS(G, s)

输入：

图 $G=(V,E)$ ，源点 s

初始化：

对每个点 $v \in V$ ：
 $visited[v] = false$
 $dist[v] = +\infty$
 $parent[v] = NIL$

$visited[s] = true$
 $dist[s] = 0$
 $parent[s] = NIL$

建立空队列 Q

将 s 入队

循环：

 当 Q 非空时：

u = 队首出队

 对每个邻居 $v \in \text{Adj}[u]$ ：

 如果 $\text{visited}[v] == \text{false}$ ：

$\text{visited}[v] = \text{true}$

$\text{dist}[v] = \text{dist}[u] + 1$

$\text{parent}[v] = u$

 将 v 入队

输出：

$\text{dist}, \text{parent}$

BFS 正确性证明模板

对层数归纳。

- 初始时 $L_0 = s$ ，距离为 0。
- 假设所有第 0 到第 k 层节点距离正确。
- BFS 从第 k 层扩展到未访问邻居，这些节点存在长度 $k + 1$ 的路径。
- 若它们有更短路径，则应更早被发现，矛盾。
- 所以第一次发现时的距离就是最短距离。

2.3 必考题型 2：计数最短路径条数

题型特征：

给无权图 G 和两个点 s, t ，求从 s 到 t 的最短路径条数，不能枚举所有路径。

核心做法：

- BFS 求层数；
- 动态维护 $\text{count}[v]$ ；
- 若从 u 到 v 是从上一层到下一层，则 $\text{count}[v] += \text{count}[u]$ 。

你的作业参考答案也指出：从 v 到某点的最短路径必须每一步层号加一；某个第 j 层节点的最短路径条数等于所有相邻的第 $j - 1$ 层节点路径条数之和。

必背伪代码：最短路径条数

算法 `Count-Shortest-Paths(G, s, t)`

初始化:

对每个点 v :

$\text{dist}[v] = +\infty$

$\text{count}[v] = 0$

$\text{dist}[s] = 0$

$\text{count}[s] = 1$

Q = 空队列

s 入队

循环:

当 Q 非空时:

u = 出队

对每个邻居 $v \in \text{Adj}[u]$:

如果 $\text{dist}[v] == +\infty$:

$\text{dist}[v] = \text{dist}[u] + 1$

$\text{count}[v] = \text{count}[u]$

v 入队

否则如果 $\text{dist}[v] == \text{dist}[u] + 1$:

$\text{count}[v] = \text{count}[v] + \text{count}[u]$

输出:

$\text{count}[t]$

正确性证明模板

按 BFS 层数归纳。

对第 j 层节点 y ，任何最短路径最后一步必来自第 $j - 1$ 层某个邻居 x 。

所以:

$$\text{count}[y] = \sum_{x \in L_{j-1}, (x,y) \in E} \text{count}[x]$$

算法正是按这个公式累加。

2.4 必考题型 3：二分图判定

题型特征

判断图是否二分；若不是，说明存在奇环。

必背伪代码：BFS 染色

算法 `Bipartite-Test(G)`

初始化：

 对每个点 v ：

$color[v] = UNCOLORED$

对每个点 $s \in V$ ：

 如果 $color[s] == UNCOLORED$ ：

$color[s] = 0$

$Q = \text{空队列}$

s 入队

 当 Q 非空时：

$u = \text{出队}$

 对每个邻居 $v \in \text{Adj}[u]$ ：

 如果 $color[v] == UNCOLORED$ ：

$color[v] = 1 - color[u]$

v 入队

 否则如果 $color[v] == color[u]$ ：

 返回 "不是二分图"

返回 "是二分图"

证明模板

BFS 分层有性质：任意边连接的两个点层数差至多 1。

若不存在同层边，则所有边都连接相邻层，按层数奇偶染色即可，图是二分图。

若存在同层边 (x, y) ，则 x, y 到其最近公共祖先的 BFS 树路径加上边 (x, y) 形成奇环，因此不是二分图。你的图论资料也使用了这个“同层边 $\Rightarrow$ 奇环”的证明结构。

2.5 必考题型 4：拓扑排序与约束一致性

题型特征：

有一些先后关系，判断是否能同时满足。
若能，输出一个顺序；若不能，报告矛盾。

常见关键词：

- prerequisite;
- before/after;
- dependency;
- schedule;
- dates;
- constraints;
- partial order。

必背伪代码：Kahn 拓扑排序

算法 Topological-Sort(G)

输入：

有向图 $G=(V,E)$

初始化：

对每个点 v ：

$\text{indeg}[v] = 0$

对每条边 $(u,v) \in E$ ：

$\text{indeg}[v] = \text{indeg}[v] + 1$

$Q =$ 所有 $\text{indeg}[v] == 0$ 的点

$\text{order} =$ 空序列

循环：

当 Q 非空时：

$u =$ 出队

将 u 加入 order

对每条边 (u,v) ：

$\text{indeg}[v] = \text{indeg}[v] - 1$

如果 $\text{indeg}[v] == 0$ ：

v 入队

判断：

如果 $|\text{order}| < |V|$ ：

返回 "有环，无拓扑序"

否则：

返回 `order`

证明模板

1. 若图有有向环，则环上每个点都有来自环内的前驱，不可能产生合法拓扑序。
2. 若算法无法输出所有点，说明剩余子图无入度为 0 的点，所以必有有向环。
3. 若输出了所有点，删除点的顺序保证每条边 (u, v) 中 u 先于 v ，所以是拓扑序。

典型建模模板：时间约束题

如果题目说：

- P_i 死于 P_j 出生前；
- P_i 和 P_j 生命周期有重叠；

建图：

- 每个人 P_i 建两个节点 b_i, d_i ；
- 加边 $b_i \rightarrow d_i$ ；
- 若 P_i 死于 P_j 出生前，加 $d_i \rightarrow b_j$ ；
- 若二者重叠，加 $b_i \rightarrow d_j$ 和 $b_j \rightarrow d_i$ 。

然后判断图是否有环。若无环，按拓扑序赋递增时间。你的作业答案正是这种建图方法。

2.6 必考题型 5：Dijkstra 与变形

Dijkstra 标准模板

算法 `Dijkstra(G, s)`

初始化：

对每个点 v ：

`dist[v] = +∞`

`parent[v] = NIL`

`visited[v] = false`

`dist[s] = 0`

循环 n 次：

在所有 `visited[v] == false` 的点中，

选择 $\text{dist}[v]$ 最小的点 u

如果 $\text{dist}[u] == +\infty$:
停止

$\text{visited}[u] = \text{true}$

对每条边 (u, v) :

如果 $\text{visited}[v] == \text{false}$ 且 $\text{dist}[u] + \text{length}(u, v) < \text{dist}[v]$:
 $\text{dist}[v] = \text{dist}[u] + \text{length}(u, v)$
 $\text{parent}[v] = u$

输出:

$\text{dist}, \text{parent}$

Dijkstra 正确性证明模板

设 S 是已经确定最短距离的点集。

不变量:

对每个 $u \in S$, $\text{dist}[u]$ 已经等于从 s 到 u 的真实最短路距离。

每次选 $V - S$ 中 dist 最小的点 v 。

若存在更短路径到 v , 该路径必须第一次从 S 跨到 $V - S$ 。由于边权非负, 跨出点的候选距离不会小于当前选中的 $\text{dist}[v]$, 矛盾。

所以 v 的距离已确定。

易错点

BFS 层数表示最少边数, 不表示最小费用。期中真题中也出现过这种陷阱: 即使 t 在 BFS 第 1 层, Dijkstra 也不一定一次迭代就确定 t , 因为 Dijkstra 按费用最小扩展, 不按边数扩展。

变形题: 最小瓶颈路径

如果路径代价不是边权和, 而是最大边权:

$$b(P) = \max_{e \in P} \ell(e)$$

则松弛公式从:

$$\text{dist}[v] = \min(\text{dist}[v], \text{dist}[u] + \ell(u, v))$$

改成：

$$b[v] = \min(b[v], \max(b[u], \ell(u, v)))$$

你的作业资料中也用类似 Dijkstra 的割性质证明最小瓶颈路径。

3. 贪心 Greedy

3.1 必须掌握的重点

贪心不是“看起来局部最优就行”。资料中强调，贪心最难的是确定局部规则是否安全，并用 stays ahead、exchange argument、structural lower bound 证明它不会破坏全局最优。

必须掌握：

1. 区间调度：最早结束优先；
 2. 区间分区：按开始时间处理，用最早空闲资源；
 3. 最小化最大延迟：最早截止时间优先；
 4. 卡车装箱：按顺序能装就装，装不下再发车；
 5. Dijkstra：每次确定当前最短距离点；
 6. MST：Kruskal、Prim，cut property、cycle property；
 7. Huffman 编码：合并两个最低频率节点；
 8. 反例意识：很多“看起来合理”的贪心是错的。
-

3.2 贪心证明总模板

模板 A：stays ahead

适合：

- 区间调度；
- 卡车装箱；
- 某些按顺序装载问题。

写法：

1. 设贪心解第 i 步达到的指标为 G_i ；
 2. 设任意最优解第 i 步达到的指标为 O_i ；
 3. 证明对所有 i ， G_i 不落后于 O_i ；
 4. 因此若最优能完成 k 步，贪心也能完成至少 k 步。
-

模板 B: exchange argument

适合:

- 最小化最大延迟;
- Huffman;
- 一些排序型贪心。

写法:

1. 取一个最优解 O ;
 2. 如果 O 不符合贪心选择, 则找到一个局部结构可交换;
 3. 证明交换后仍可行, 且目标值不变差;
 4. 重复交换, 得到一个包含贪心选择的最优解;
 5. 因此贪心选择安全。
-

模板 C: 结构下界

适合:

- 区间分区;
- MST;
- 某些资源最少问题。

写法:

1. 先证明任何解都至少需要 L ;
 2. 再证明贪心算法使用恰好 L ;
 3. 所以贪心最优。
-

3.3 必考题型 1: 区间调度 Interval Scheduling

题型

给若干区间 $[s_i, f_i]$, 选最多数量的互不重叠区间。

必背伪代码

```
算法 Interval-Scheduling(intervals)
```

输入：

`intervals = {1,...,n}`

每个区间 `i` 有开始时间 `s[i]` 和结束时间 `f[i]`

步骤：

按 `f[i]` 从小到大排序

`A` = 空集合

`current_end` = $-\infty$

按排序后顺序扫描每个区间 `i`：

如果 `s[i] >= current_end`：

将 `i` 加入 `A`

`current_end` = `f[i]`

输出：

`A`

正确性证明：stays ahead

设贪心选出的区间为：

$i_1, i_2, \dots, i_k$

任意最优解选出的区间为：

$j_1, j_2, \dots, j_m$

按照结束时间顺序排列。

证明：

$f(i_r) \leq f(j_r)$ 对每个 r 成立。

- $r = 1$ ：贪心选择全体中结束最早的区间，所以成立。
- 假设 $f(i_{r-1}) \leq f(j_{r-1})$ 。
- 因为 j_r 与 j_{r-1} 相容，所以 $s(j_r) \geq f(j_{r-1}) \geq f(i_{r-1})$ 。
- 因此 j_r 在贪心第 r 步时可选。
- 贪心选结束最早的可选区间，所以 $f(i_r) \leq f(j_r)$ 。

若最优解比贪心多，则 j_{k+1} 在贪心结束后仍可选，矛盾。

所以贪心最优。

你的资料中也把这个证明称为本章最经典的 stays ahead 模板。

3.4 必考题型 2：区间分区 Interval Partitioning

题型

所有区间都要安排，求最少资源数。

例如最少教室数、最少机器数。

必背伪代码

算法 Interval-Partitioning(intervals)

输入：

每个区间 i 有开始时间 $s[i]$ 和结束时间 $f[i]$

步骤：

按 $s[i]$ 从小到大排序

建立最小堆 H

堆中元素为 (某资源当前结束时间, 资源编号)

$resource_count = 0$

对每个区间 i ，按开始时间顺序：

如果 H 非空 且 堆顶结束时间 $\leq s[i]$ ：

取出堆顶 (end, r)

将区间 i 分配给资源 r

将 $(f[i], r)$ 放回堆中

否则：

$resource_count = resource_count + 1$

$r = resource_count$

将区间 i 分配给新资源 r

将 $(f[i], r)$ 放入堆中

输出：

$resource_count$ 和分配方案

正确性证明：结构下界

定义 depth：

某一时刻同时进行的区间最大数。

任何方案至少需要 $depth$ 个资源。

设贪心用了 d 个资源。考虑第 d 个资源第一次被打开时的区间 i 。

此时前 $d - 1$ 个资源都无法接纳 i ，说明它们的最后区间都与 i 重叠。

所以存在某个时刻有至少 d 个区间重叠。

因此：

$$depth \geq d$$

又因为任何方案至少需要 $depth$ 个资源，贪心用了 d 个资源，所以贪心最优。

3.5 必考题型 3：最小化最大延迟

题型

每个任务有：

- processing time t_i ；
- deadline d_i 。

完成时间 C_i ，延迟：

$$L_i = \max(0, C_i - d_i)$$

目标：

$$\min \max_i L_i$$

必背伪代码：EDF

算法 Minimize-Maximum-Lateness(jobs)

输入：

每个作业 i 有处理时间 $t[i]$ 和截止时间 $d[i]$

步骤：

按 $d[i]$ 从小到大排序

$time = 0$

$schedule =$ 空序列

按排序后顺序扫描每个作业 i ：

将 i 加入 $schedule$

$time = time + t[i]$

```
C[i] = time
L[i] = max(0, C[i] - d[i])
```

输出:

schedule 和 $\max_i L[i]$

正确性证明: exchange argument

1. 先说明存在一个无空闲时间的最优调度。
2. 若某个最优调度中存在相邻逆序作业 i, j , 满足 $d_i > d_j$, 但 i 排在 j 前。
3. 交换 i, j 。
4. 交换后不影响其他作业完成时间。
5. 只需比较 i, j 的最大延迟。
6. 可以证明交换后最大延迟不增加。
7. 不断消除逆序, 得到按 deadline 排序的最优调度。

所以 EDF 最优。

3.6 必考题型 4: 按顺序装车/装箱

题型

包裹按顺序到达, 每辆车容量 W 。必须按顺序运输。能装就装, 下一个装不下就发车。证明贪心用车数最少。

必背伪代码

算法 Greedy-Truck-Packing($w[1..n]$, W)

初始化:

```
trucks = 空列表
current_truck = 空
current_weight = 0
```

对 $i = 1$ 到 n :

```
如果 current_weight + w[i] <= W:
    将包裹 i 放入 current_truck
    current_weight = current_weight + w[i]
```

否则：

发出 `current_truck`，将其加入 `trucks`

`current_truck = {i}`

`current_weight = w[i]`

循环结束后：

如果 `current_truck` 非空：

将 `current_truck` 加入 `trucks`

输出：

`trucks`

正确性证明：stays ahead

设贪心第 i 辆车装完后，最后装到的包裹编号为 $f(i)$ 。

设任意其他方案 H 第 i 辆车装完后，最后装到的包裹编号为 $g(i)$ 。

证明：

$$f(i) \geq g(i)$$

对所有 i 成立。

基础情况：第一辆车贪心尽量多装，所以 $f(1) \geq g(1)$ 。

归纳步骤：假设 $f(i) \geq g(i)$ 。第 $i+1$ 辆车中，贪心从不晚于 H 的位置开始装，并且一直装到再多一个会超重。因此 H 不可能在第 $i+1$ 辆车装得比贪心更远。

所以对所有 i ，贪心都不落后。

若某方案用 k 辆车装完，则贪心用 k 辆也能装完。因此贪心车数最少。

这个题在你的作业中明确要求用 interval scheduling 类似的 stays ahead 思路证明。

3.7 必考题型 5：MST 最小生成树

Kruskal 伪代码

算法 `Kruskal(G)`

输入：

连通无向图 $G=(V,E)$ ，每条边有权重 $w(e)$

初始化：

```
A = 空集合
对每个点 v 建立一个独立集合 Make-Set(v)
```

步骤:

按边权从小到大排序所有边

对每条边 $e=(u,v)$, 按权重从小到大扫描:

如果 $\text{Find}(u) \neq \text{Find}(v)$:

将 e 加入 A

$\text{Union}(u,v)$

如果 $|A| == |V|-1$:

停止

输出:

A

Kruskal 正确性证明: cut property

维护不变量:

当前边集 A 是某棵 MST 的子集。

每次 Kruskal 选择连接两个不同连通分量的最轻边 e 。

考虑由当前某个连通分量与其余点构成的割。 e 是跨越该割的最轻边。

根据 cut property, 跨越任意割的最轻边是安全边, 可以加入某棵 MST。

所以不变量保持。

最终 A 有 $n - 1$ 条边且连通, 是 MST。

Prim 伪代码

算法 $\text{Prim}(G, s)$

初始化:

$S = \{s\}$

$A = \text{空集合}$

循环:

当 $S \neq V$ 时:

在所有一端在 S 、另一端在 $V-S$ 的边中,

选择权重最小的边 $e=(u,v)$, 其中 $u \in S, v \notin S$

将 e 加入 A

将 v 加入 S

输出：

A

证明同样用 cut property。

3.8 贪心题考场判断方法

看到贪心题，先问四个问题：

1. 目标是最大数量、最小资源、最早完成、最小最大值，还是最小总成本？
2. 有没有天然排序指标？结束时间、开始时间、deadline、权重、频率？
3. 证明应该用哪一种？
 - 每步“领先” → stays ahead；
 - 任意最优解可调整 → exchange；
 - 有明显下界 → structural lower bound；
 - 图上跨割选边 → cut property。
4. 能不能构造反例推翻其他看似合理规则？

4. 分治 Divide and Conquer

4.1 必须掌握的重点

分治法不是简单递归，而是：

1. Divide：拆成子问题；
2. Conquer：递归求解；
3. Combine：合并答案。

资料中也强调：分治的关键不只是“拆”，更是“合”。如果 combine 太慢，分治不会变快。

必须掌握：

1. 递归式分析；
2. 归并排序；
3. 计数逆序；
4. 最近点对；
5. Karatsuba；

6. 选择问题/median-of-medians;
7. 二分式查询;
8. 分治正确性证明。

4.2 分治题通用答题模板

算法 `Divide-and-Conquer(X)`

如果 X 的规模足够小:
直接求解并返回

将 X 分成若干子问题 $X_1, X_2, \dots, X_k$

对子问题递归:

```
ans1 = Solve(X1)
ans2 = Solve(X2)
...
ansk = Solve(Xk)
```

合并:

```
ans = Combine(ans1, ans2, ..., ansk)
```

返回 `ans`

正确性证明模板

对规模 n 归纳。

- 基础情况：小规模时直接求解正确。
- 归纳假设：所有规模小于 n 的子问题都能正确求解。
- 当前规模 n ：
 - divide 保证子问题覆盖所有可能情况;
 - conquer 根据归纳假设正确;
 - combine 能把子问题答案正确合成原问题答案。
- 所以算法正确。

复杂度模板

写出递推式:

$T(n) = aT(n/b) + f(n)$

然后分析：

递推式	复杂度
$T(n) = 2T(n/2) + O(n)$	$O(n \log n)$
$T(n) = T(n/2) + O(n)$	$O(n)$
$T(n) = 2T(n/2) + O(n^2)$	$O(n^2)$
$T(n) = 3T(n/2) + O(n)$	$O(n^{\log_2 3})$
$T(n) = 4T(n/2) + O(n)$	$O(n^2)$

资料中也把这些递推式作为分治复杂度分析核心，并说明每层工作量相同、底层主导、根部主导三种情况。

4.3 必考题型 1：归并排序 Merge Sort

伪代码

```
算法 Merge-Sort(A)

如果 |A| <= 1:
    返回 A

将 A 分成左右两半:
    L = A 的左半部分
    R = A 的右半部分

递归排序:
    L_sorted = Merge-Sort(L)
    R_sorted = Merge-Sort(R)

合并:
    返回 Merge(L_sorted, R_sorted)
```

Merge 伪代码

```
算法 Merge(L, R)

i = 1, j = 1
B = 空数组
```

当 $i \leq |L|$ 且 $j \leq |R|$ 时:

 如果 $L[i] \leq R[j]$:

 将 $L[i]$ 加入 B

$i = i + 1$

 否则:

 将 $R[j]$ 加入 B

$j = j + 1$

将 L 中剩余元素加入 B

将 R 中剩余元素加入 B

返回 B

正确性证明

- 左右递归根据归纳假设已经有序;
- Merge 能把两个有序数组合成一个有序数组;
- 所以整体有序。

复杂度:

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

4.4 必考题型 2: 计数逆序 Counting Inversions

题型

给数组 $A[1..n]$, 求逆序对数量:

(i, j) 满足 $i < j$ 且 $A[i] > A[j]$ 。

核心思想

逆序分三类:

1. 左半内部逆序;
2. 右半内部逆序;
3. 跨左右两半逆序。

前两类递归求。

跨半逆序在 merge 时顺便数。

必背伪代码

算法 Sort-and-Count(A)

如果 $|A| \leq 1$:
 返回 (A, 0)

将 A 分成 L 和 R

(L_sorted, count_L) = Sort-and-Count(L)
(R_sorted, count_R) = Sort-and-Count(R)

(M_sorted, count_split) = Merge-and-Count(L_sorted, R_sorted)

返回:
 (M_sorted, count_L + count_R + count_split)

算法 Merge-and-Count(L, R)

i = 1, j = 1
B = 空数组
count = 0

当 $i \leq |L|$ 且 $j \leq |R|$ 时:

 如果 $L[i] \leq R[j]$:
 将 $L[i]$ 加入 B
 i = i + 1

 否则:
 将 $R[j]$ 加入 B
 count = count + ($|L| - i + 1$)
 j = j + 1

将 L 剩余元素加入 B
将 R 剩余元素加入 B

返回 (B, count)

正确性证明模板

按规模归纳。

- 左半、右半逆序由递归正确计算。

- 合并时，若 $L[i] > R[j]$ ，由于 L 已有序，所以 $L[i], L[i+1], \dots, L[|L|]$ 全部大于 $R[j]$ 。
- 因此一次新增 $|L| - i + 1$ 个跨半逆序。
- 三类逆序互不重叠且覆盖全部逆序。
- 所以计数正确。

你的资料中也强调，核心语句是 `count += len(L)-i`，它使跨半逆序被批量统计，从而把 $O(n^2)$ 降到 $O(n \log n)$ 。

4.5 必考题型 3：最近点对 Closest Pair

题型

给平面上 n 个点，找距离最近的一对点。

朴素：

$O(n^2)$

分治：

$O(n \log n)$

核心思路

1. 按 x 坐标分成左右两半；
2. 递归求左半最近距离 δ_L ；
3. 递归求右半最近距离 δ_R ；
4. 令 $\delta = \min(\delta_L, \delta_R)$ ；
5. 只需检查中线附近宽度 2δ 的 strip；
6. strip 中按 y 坐标排序，每个点只需检查后面常数个点；
7. 返回最小值。

资料中说明，最近点对难点是跨边点对；算法用 δ 限制 strip，再用几何装箱论证每个点只需检查 $O(1)$ 个后继点，从而合并为 $O(n)$ 。

伪代码框架

算法 Closest-Pair(P)

预处理：

$P_x = P$ 按 x 坐标排序

$P_y = P$ 按 y 坐标排序

返回 $\text{Closest-Rec}(P_x, P_y)$

算法 $\text{Closest-Rec}(P_x, P_y)$

如果 $|P| \leq 3$:

暴力检查所有点对

返回最近点对及距离

按 P_x 中位数 x_{mid} 将点分成:

Q = 左半点集

R = 右半点集

通过扫描 P_y 得到:

Q_y = Q 中点按 y 排序

R_y = R 中点按 y 排序

递归:

$(\text{pair}_L, \text{delta}_L) = \text{Closest-Rec}(Q_x, Q_y)$

$(\text{pair}_R, \text{delta}_R) = \text{Closest-Rec}(R_x, R_y)$

令:

$\text{delta} = \min(\text{delta}_L, \text{delta}_R)$

best_pair = 对应较小的一对

构造 strip :

$S = P_y$ 中满足 $|x - x_{\text{mid}}| < \text{delta}$ 的点

按 S 的 y 顺序扫描:

对每个点 $S[i]$:

只检查后面常数个点 $S[i+1], \dots, S[i+c]$

若距离更小, 则更新 best_pair 和 delta

返回 $\text{best_pair}, \text{delta}$

正确性证明模板

最近点对只可能有三种情况:

1. 两点都在左半;
2. 两点都在右半;
3. 一点在左半, 一点在右半。

前两种由递归正确处理。

第三种如果距离小于 δ ，则两点横坐标距离必小于 δ ，所以都在 strip 中。

在 strip 中，几何装箱论证保证每个点只需检查后面常数个点。

所以所有可能比 δ 更优的跨半点对都被检查到。

因此算法正确。

复杂度：

$$T(n) = 2T(n/2) + O(n) = O(n \log n)$$

注意：如果每层重新按 y 排序，会变成 $O(n \log^2 n)$ 。要维护 P_y ，用线性扫描分出 Q_y, R_y 。

4.6 必考题型 4：分治递推式证明

题型

给递推式：

$$T(n) \leq T(\alpha n) + T(\beta n) + n$$

证明：

- 若 $\alpha + \beta < 1$ ，则 $T(n) = O(n)$ ；
 - 若 $\alpha + \beta = 1$ ，则 $T(n) = O(n \log n)$ 。
-

证明模板 1： $\alpha + \beta < 1$

猜：

$$T(n) \leq cn$$

代入：

$$T(n) \leq c\alpha n + c\beta n + n$$

$$= [c(\alpha + \beta) + 1]n$$

要使其 $\leq cn$ ，只需：

$$c(\alpha + \beta) + 1 \leq c$$

即：

$$c \geq \frac{1}{1-\alpha-\beta}$$

取足够大 c ，归纳成立。

证明模板 2: $\alpha + \beta = 1$

猜:

$$T(n) \leq cn \log n$$

代入:

$$T(n) \leq c\alpha n \log(\alpha n) + c\beta n \log(\beta n) + n$$

展开:

$$= c(\alpha + \beta)n \log n + cn(\alpha \log \alpha + \beta \log \beta) + n$$

由于 $\alpha + \beta = 1$, 且 $\alpha \log \alpha + \beta \log \beta < 0$, 取足够大 c 可抵消 $+n$ 。

所以:

$$T(n) \leq cn \log n$$

4.7 必考题型 5: Karatsuba 整数乘法

核心公式

将:

$$x = 2^{n/2}x_L + x_R$$

$$y = 2^{n/2}y_L + y_R$$

朴素分治要算四个乘法:

$$x_L y_L, x_L y_R, x_R y_L, x_R y_R$$

Karatsuba 只算三个:

$$A = x_L y_L$$

$$C = x_R y_R$$

$$B = (x_L + x_R)(y_L + y_R)$$

中间项:

$$x_L y_R + x_R y_L = B - A - C$$

所以:

$$xy = 2^n A + 2^{n/2}(B - A - C) + C$$

递推式:

$$T(n) = 3T(n/2) + O(n)$$

所以：

$$T(n) = O(n^{\log_2 3})$$

资料中也强调，分治不一定自动变快，关键是减少递归次数；4 个半规模乘法仍是 $O(n^2)$ ，3 个才得到 Karatsuba 的改进。

5. 考前必背题型清单

下面这张表建议直接背。

模块	题型	算法	证明关键词
稳定匹配	基础 stable matching	Gale-Shapley	终止、完美、无 blocking pair
稳定匹配	ties	打破并列 + GS / 反例	strong 保留严格偏好；weak 反例
稳定匹配	现实问题规约	构造双方偏好	若非法则形成 blocking pair
图	无权最短路	BFS	层号 = 最短距离
图	最短路径条数	BFS + count	第 j 层来自第 $j - 1$ 层
图	二分图	BFS 染色	同层边 奇环
图	先后约束	拓扑排序	DAG 有拓扑序，有环则矛盾
图	带权最短路	Dijkstra	已确定集合 S 的不变量
贪心	区间调度	最早结束	stays ahead
贪心	区间分区	最早开始 + 最早空闲资源	depth 下界
贪心	最大延迟最小	最早 deadline	exchange argument
贪心	顺序装车	能装就装	stays ahead
贪心	MST	Kruskal / Prim	cut property / cycle property
分治	排序	Merge Sort	归纳 + merge 正确
分治	逆序数	Sort-and-Count	跨半逆序批量统计
分治	最近点对	Divide + strip	跨边点对限制在 strip
分治	递推式	递归树/归纳	每层工作量
分治	Karatsuba	3 个子乘法	$T(n) = 3T(n/2) + O(n)$

6. 考场写证明的固定句式

6.1 Gale-Shapley 稳定性句式

假设输出匹配存在 blocking pair (a, b) 。

因为 a 更喜欢 b 胜过当前对象，所以 a 一定曾向 b 求婚。

当时 b 要么拒绝 a ，要么后来抛弃 a ，两种情况都说明 b 最终对象不差于 a 。

这与 b 更喜欢 a 矛盾。

所以不存在 blocking pair。

6.2 BFS 最短路句式

BFS 按层扩展。第 0 层是源点。

若所有第 0 到第 k 层均为真实距离，则从第 k 层首次发现的点有长度 $k + 1$ 的路径。

若存在更短路径，它应在更早层被发现，矛盾。

因此层号等于最短距离。

6.3 拓扑排序句式

若图有环，则环上每个点都有前驱，不可能存在拓扑序。

若图无环，则必有入度为 0 的点。算法反复删除入度为 0 的点。

输出序列中每条边的起点都先于终点，因此是拓扑序。

6.4 贪心 stays ahead 句式

设贪心解第 i 步指标为 G_i ，任意最优解第 i 步指标为 O_i 。

证明对所有 i ， G_i 至少不劣于 O_i 。

基础情况由贪心第一步选择成立。

归纳步骤中，由归纳假设可知最优解第 i 个选择在贪心第 i 步仍可行；贪心选择当前最优，所以继续领先。

因此贪心不可能比最优解差。

6.5 贪心 exchange argument 句式

取一个最优解 O ，在所有最优解中选择与贪心解前缀相同最长的一个。

若它下一步与贪心选择不同，则将该部分替换/交换为贪心选择。

证明替换后仍可行，且目标值不变差。

这得到一个与贪心前缀更长的最优解，矛盾。

因此存在包含贪心选择的最优解。

6.6 分治正确性句式

对输入规模 n 归纳。

基础情况直接求解正确。

对规模 n 的问题，算法将其分为若干规模更小的子问题。

根据归纳假设，递归调用返回的子问题答案正确。

合并步骤枚举/覆盖了所有跨子问题的可能情况，并正确选择最优/统计总数。

因此原问题答案正确。

7. 最后考前建议

你现在优先背这四组“核心模板”：

1. **GS 模板**：伪代码 + 终止性 + 完美性 + 稳定性；
2. **BFS/拓扑模板**：BFS 层证明、计数最短路、拓扑排序判环；
3. **贪心三证明**：stays ahead、exchange、lower bound；
4. **分治三件套**：递归伪代码、归纳正确性、递推式复杂度。

考试写算法设计题时，答案尽量固定成：

建模/观察 → 算法伪代码 → 正确性证明 → 复杂度分析

这四块齐了，通常就不会失大分。